Midterm Experiment

Decoders, BCD Addition, Comparators, Multiplexers and a Multiplier

Student	Seyed Kiarash Eshraghi Naeini (403101383)
Instructor	Dr. Mohammad Reza Movahhedin Teaching assistant: Hossein Moghim
Semester	Fall 2025 session: 7–8 December 2025
Platform	Zynq-7010 (XC7Z010-CLG400-1), 50 MHz clock, Vivado 2019.1

Summary

This midterm experiment (between Experiments 4 and 6) implements a set of standard combinational building blocks on the FPGA: a 3-to-8 decoder, a one-digit BCD adder with input validation, 4-bit magnitude comparators (once behaviourally and once from the status flags of a subtractor), an 8-to-1 multiplexer and a 4×4 multiplier. Because all designs have at most eight inputs, each was verified *exhaustively* for this report. This found one error in the flag-based signed comparator (28 of 256 input pairs) and two wiring errors in the second bonus design; all are analysed and corrected below, and the bonus now works as a 2-bit comparator. Code: `https://github.com/seyedkiarash/seyedkiarash.github.io/tree/main/labs/logic-circuits-lab/lab04m-midterm-decoders-comparators`.

Inputs and outputs follow the extension board: the operands are `A = DIP[3:0]` and `B = DIP[7:4]`; results appear on `LED[9:0]` and the seven-segment display; the four board push buttons are *active-low* and are inverted in the designs.

1 Part 1 – 3-to-8 decoder

Each output is the minterm of the 3-bit input, e.g. $\text{show}_5 = n_2\,\overline{n_1}\,n_0$, so exactly one of `LED[7:0]` is lit – the one selected by `DIP[2:0]`. Exhaustive test: all 256 DIP settings give the correct one-hot output.

2 Part 2 – BCD adder

2.1 Design

The two digits are added by a 4-bit ripple-carry adder (four full adders built from half adders). The binary sum S (0–18) is then converted to BCD:

- if the adder produced a carry ($S \geq 16$): tens = 1, units = $(16 + S_{3:0}) - 10$;
- else if $S \geq 10$: tens = 1, units = $S - 10$;
- otherwise tens = 0, units = S.

Subtracting 10 from a 5-bit value is equivalent to the familiar "add 6 and drop the carry" correction. A helper `check10` flags operands that are not BCD digits, $A \geq 10 \iff A_3(A_2 + A_1)$; an invalid input shows `E`, and if both are invalid a second digit shows `E` as well. The units digit is shown on the display and the tens on `LED[0]` (Fig. 1).



Figure 1. BCD adder output for three inputs.

2.2 Verification

All 256 input pairs: the 100 valid digit pairs give the correct units digit and tens LED, and the 156 pairs with an invalid digit show `E` on one or two digits as specified. 0 errors.

3 Part 3 – 4-bit comparators

Three outputs are produced: $A > B$ unsigned (`LED[0]`), $A > B$ signed in two's complement (`LED[1]`) and $A = B$ (`LED[2]`).

3.1 3a – Behavioural description

The unsigned and equality tests use the Verilog operators. The signed test first compares the sign bits – if they differ, the non-negative operand is larger – and otherwise compares the remaining three bits, which order two's-complement numbers of equal sign correctly. All 256 pairs: 0 errors.

3.2 3b – Comparison from subtractor flags

The second version computes $A - B = A + \overline{B} + 1$ with the adder and derives the comparisons from the flags Z (zero), C (carry, i.e. no borrow), N (sign of the result) and V (overflow):

$$A > B \text{ (unsigned)} = C\,\overline{Z}, \qquad A > B \text{ (signed)} = \overline{Z}\,\overline{(N \oplus V)}, \qquad A = B = Z.$$

Finding. The submitted design computed V with the overflow formula of an *addition* of A and B, $V = A_3 B_3 \overline{S_3} + \overline{A_3}\,\overline{B_3} S_3$, although the adder actually adds A and $\overline{B}$. Its signed output was also built from a different expression,

$$N\,\overline{(N \oplus V)}\,\overline{V} \;+\; \overline{N}\,C\,\overline{Z} \;+\; \overline{A_3}\,B_3,$$

whose first term is always 0. Exhaustive simulation of the submitted code gave the wrong answer for **28 of the 256** pairs – exactly the pairs with $A < 0 < B$ whose difference underflows, $A - B < -8$ (e.g. $A = -8$, $B = 1$ was reported as "greater"). The corrected design uses the subtraction overflow $V = A_3 \overline{B_3}\,\overline{S_3} + \overline{A_3} B_3 S_3$ and the standard condition above; it now matches the behavioural comparator on all 256 pairs.

4 Part 4 – 8-to-1 multiplexer

The DIP switches are the eight data inputs and the inverted buttons `button[2:0]` the select code; the selected bit is shown on `LED[0]`. The submitted version described the selection with a chain of `if…else if` statements; it is written here as `assign f = A[s]`, which is equivalent and cannot infer a latch. Exhaustive test: 256 data words × 8 select codes, 0 errors.

5 Bonus 1 – 4×4 combinational multiplier

The product is formed by shift-and-add: for every bit $B_i = 1$ the partial product $A \cdot 2^i$ is added, $P = \sum_i B_i\,(A \ll i)$, giving an 8-bit result (maximum $15 \times 15 = 225$) on `LED[7:0]`. All 256 products are correct. Synthesis maps the four conditional additions to 23 LUTs and carry logic.

6 Bonus 2 – A 2-bit comparator from multiplexers

6.1 Idea

The bonus builds a comparator for two 2-bit numbers $X = \{x_1, x_0\}$ and $Y = \{y_1, y_0\}$ (from the inverted buttons: $x_0 = n_0$, $y_0 = n_1$, $x_1 = n_2$, $y_1 = n_3$) using *only* 8-to-1 multiplexers, each used as a three-input look-up table: the constant on the data inputs is the truth table and the three signals on the select inputs are the variables. As in Part 3b the comparison is done by subtraction $X - Y$:

- two multiplexers with the table `10010110` (three-input XOR) give the difference bits $d_0 = x_0 \oplus y_0$ and $d_1 = x_1 \oplus y_1 \oplus b_0$;
- two multiplexers with the table of the *borrow* function, $b_{\text{out}} = \overline{x}\,y + \overline{(x \oplus y)}\,b_{\text{in}}$, i.e. `10001110` for select $\{x, y, b_{\text{in}}\}$, give the borrow chain b_0 and b_1;
- $X < Y = b_1$, $\quad X = Y = \overline{d_0}\,\overline{d_1}$, $\quad X \geq Y = \overline{b_1}$.

6.2 Findings in the submitted version and correction

- The select buses of the borrow multiplexers were declared without a width, `wire s1 = {nb[2], nb[0], Cin};`, which in Verilog declares a *1-bit* wire: only the last element survived, so the borrow chain (and `LED[0]`) did not depend on the buttons at all.
- The borrow multiplexers paired the operand bits as (n_2, n_0) and (n_3, n_1), whereas the XOR multiplexers paired them as (n_0, n_1) and (n_2, n_3). An exhaustive search over all 256 possible tables showed that no table makes this wiring a comparator.
- The borrow table was taken from the DIP switches. With the pairing corrected, exactly one setting works – `1000 1110`, the borrow function above – which confirms the intended design.

The corrected design declares the select buses as 3-bit vectors, uses the same bit pairing in both chains and stores the borrow table as a constant, so it works without a particular DIP setting. Exhaustive test: all 16 operand pairs give the correct $\geq$, $=$, $<$ outputs (checked for 16 different DIP settings to confirm the DIP switches no longer matter).

7 Implementation results

Table 1. Resources after synthesis (Yosys, Xilinx 7-series mapping).

Design	LUTs	Note
Part 1 – 3-to-8 decoder	8	
Part 2 – BCD adder	20	
Part 3a / 3b – comparators	10 / 10	
Part 4 – 8-to-1 multiplexer	3	
Bonus 1 – multiplier	23	
Bonus 2 – 2-bit comparator	3	one LUT per output

8 Verification summary

Testbench	What is checked (exhaustive)	Result
`tb_part1`	256 DIP settings, one-hot output	**PASS**
`tb_part2`	100 valid digit pairs, 156 invalid inputs	**PASS**
`tb_part3`	256 pairs × 3 comparisons	**PASS**
`tb_part52`	256 pairs × 3 comparisons (corrected flags)	**PASS**
`tb_part4`	256 data words × 8 select codes	**PASS**
`tb_multiplier`	all 256 products	**PASS**
`tb_bonus2`	all 16 operand pairs, $\geq$ / $=$ / $<$	**PASS**

9 Conclusion

The standard combinational blocks map to a handful of LUTs each. Comparing two implementations of the same function – behavioural and flag-based – and testing them exhaustively was decisive: the flag-based signed comparison is correct only with the overflow flag of the operation actually performed, and the only failing inputs were the 28 overflow cases. The bonus shows that multiplexers used as look-up tables can build any logic function, and how an undeclared vector width silently changes a circuit.